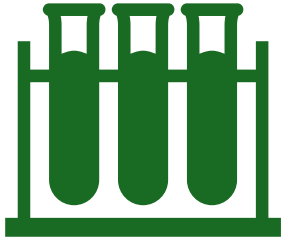
Title: Leveraging Hybrid Algorithms for Advanced Classification of Portable Water

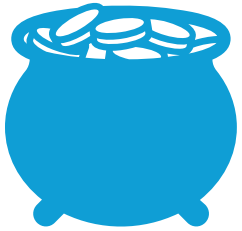
- Subtitle: A Comprehensive Data Science Approach
- Presented by:
- Pelleti Abhigna[22BCE3855]
- Saniya[22BCE2208]
- Mannyak[22BCE3056]

Problem Statement

Access to potable water remains a challenge in many urban areas despite technological advancements.



Conventional water quality testing is time-consuming and requires laboratory setups.



Need for a reliable, real-time, and intelligent system to predict water potability accurately.

Proposed Solution

Development of a hybrid model combining Neural Networks, XGBoost, Gradient Boosting, Logistic Regression, and Random Forest classifiers.

Using SMOTE to tackle class imbalance.

Leveraging data science techniques such as PCA, Polynomial Features, and KNN Imputer to preprocess data.

Integration into a web-based system for user-friendly interaction.

Abstract

- This project explores machine learning techniques to assess water potability.
- The proposed model integrates several classifiers and preprocessing steps for optimal prediction.
- Results in a scalable and practical solution for smart city water quality monitoring.

Introduction

IMPORTANCE OF CLEAN DRINKING WATER.

OVERVIEW OF CURRENT CHALLENGES IN WATER QUALITY TESTING.

INTRODUCTION TO SMART CITIES AND THE NEED FOR INTELLIGENT SYSTEMS.

BRIEF INTRO TO MACHINE LEARNING AND ITS ROLE.

Literature Survey

Review of past methods for water quality testing.

Comparison of traditional and AI-based systems.

Limitations of individual classifiers.

Methodology

Dataset: Water Potability dataset with parameters like pH, Hardness, Solids, etc.

Imputation: KNN Imputer to handle missing data. - Balancing: SMOTE for class imbalance.

Feature Engineering: Polynomial Features. - Dimensionality Reduction: PCA (Principal Component Analysis).

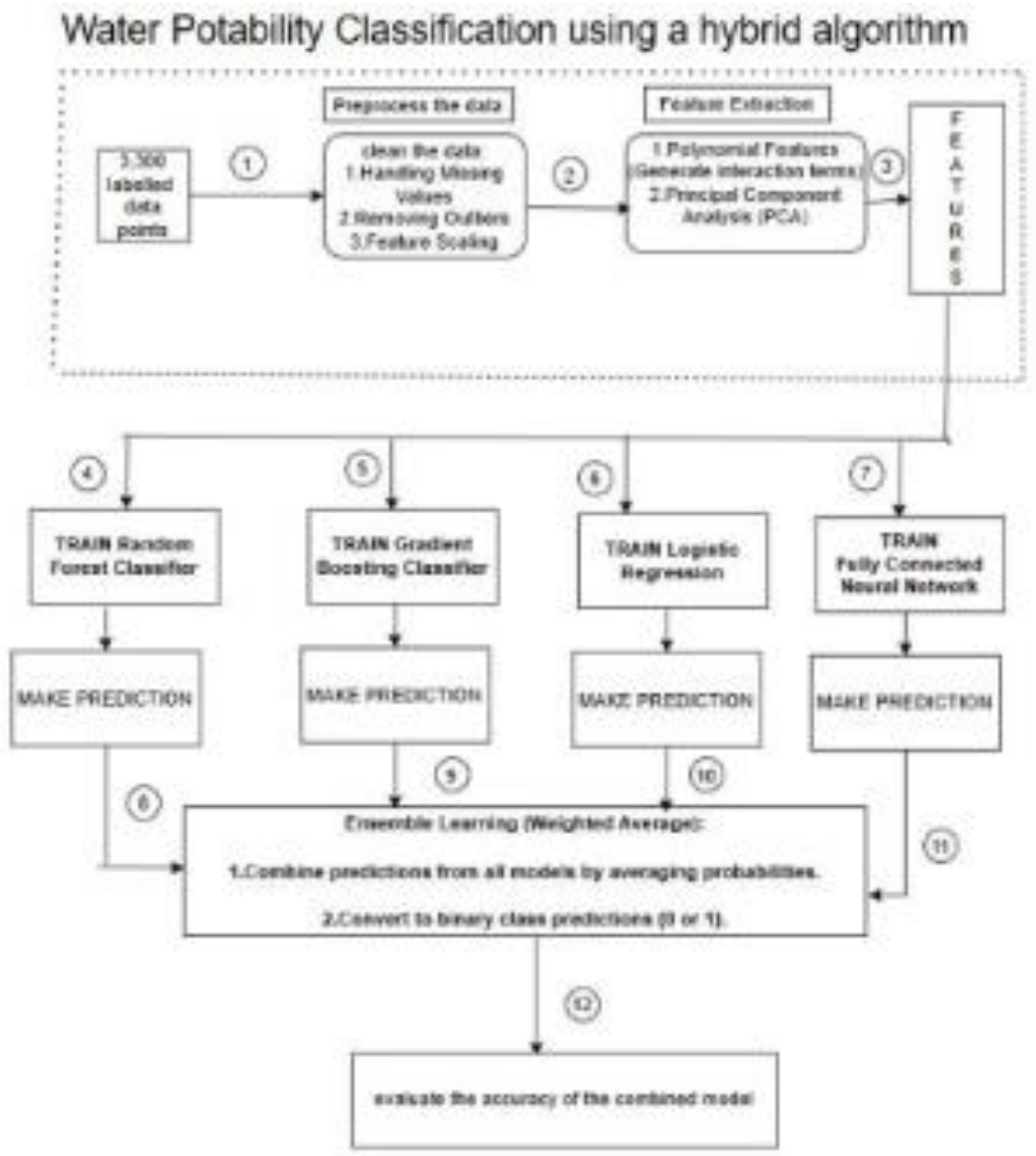
Classifiers Used

- Neural Network (Keras)
- Logistic Regression
- Random Forest
- Gradient Boosting
- XGBoost
- Voting Ensemble for combining predictions

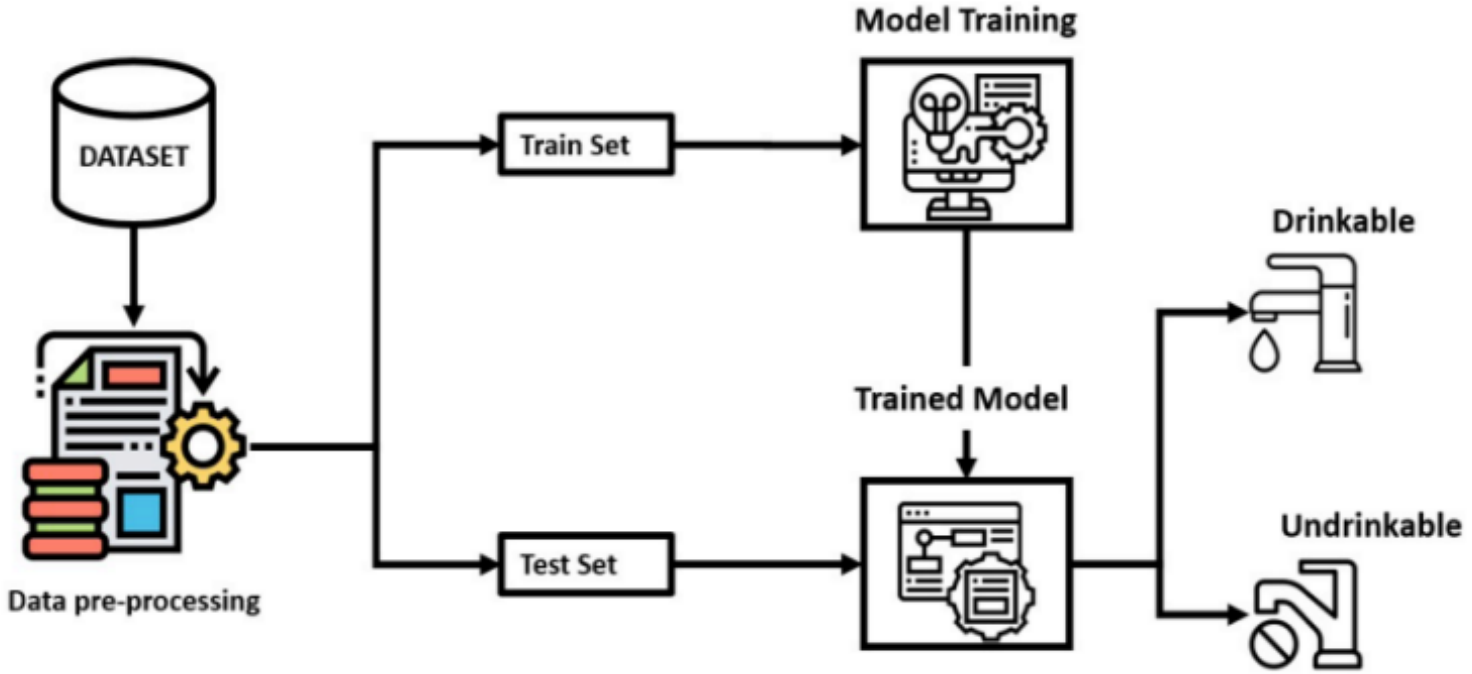
Neural Network Architecture

- Dense(1024) + ReLU + Dropout + BatchNorm
- Dense(512) + ReLU + Dropout + BatchNorm
- Dense(256) + ReLU + Dropout + BatchNorm
- Dense(128) + ReLU + Dropout
- Output: Dense(1, Sigmoid)

System Architecture



Simplified Architecture



Requirement Specifications

Software Requirements:

1.Data processing tools:

- Python packages with data processing modules (probably NumPy, Pandas)
- Data pre-processing tools (KNN Imputer for missing data)
- SMOTE for imbalanced classes

2.Machine Learning Framework:

- Implementations of statistical models (Random Forest, Logistic Regression)
- libraries for gradient boosting and XGBoost
- if you are doing deep learning, you will probably have a model (probably TensorFlow or PyTorch for neural networks)

3.Analysis and visualization package:

- libraries for correlation analysis
- libraries for confusion matrices
- libraries for ROC curves and precision-recall curves
- libraries for feature importance visualization

4. Model evaluation package:

- libraries for cross validation
- metrics for evaluation (accuracy, precision, recall, F1-score), and
- ways to implement ensemble methods for the models.

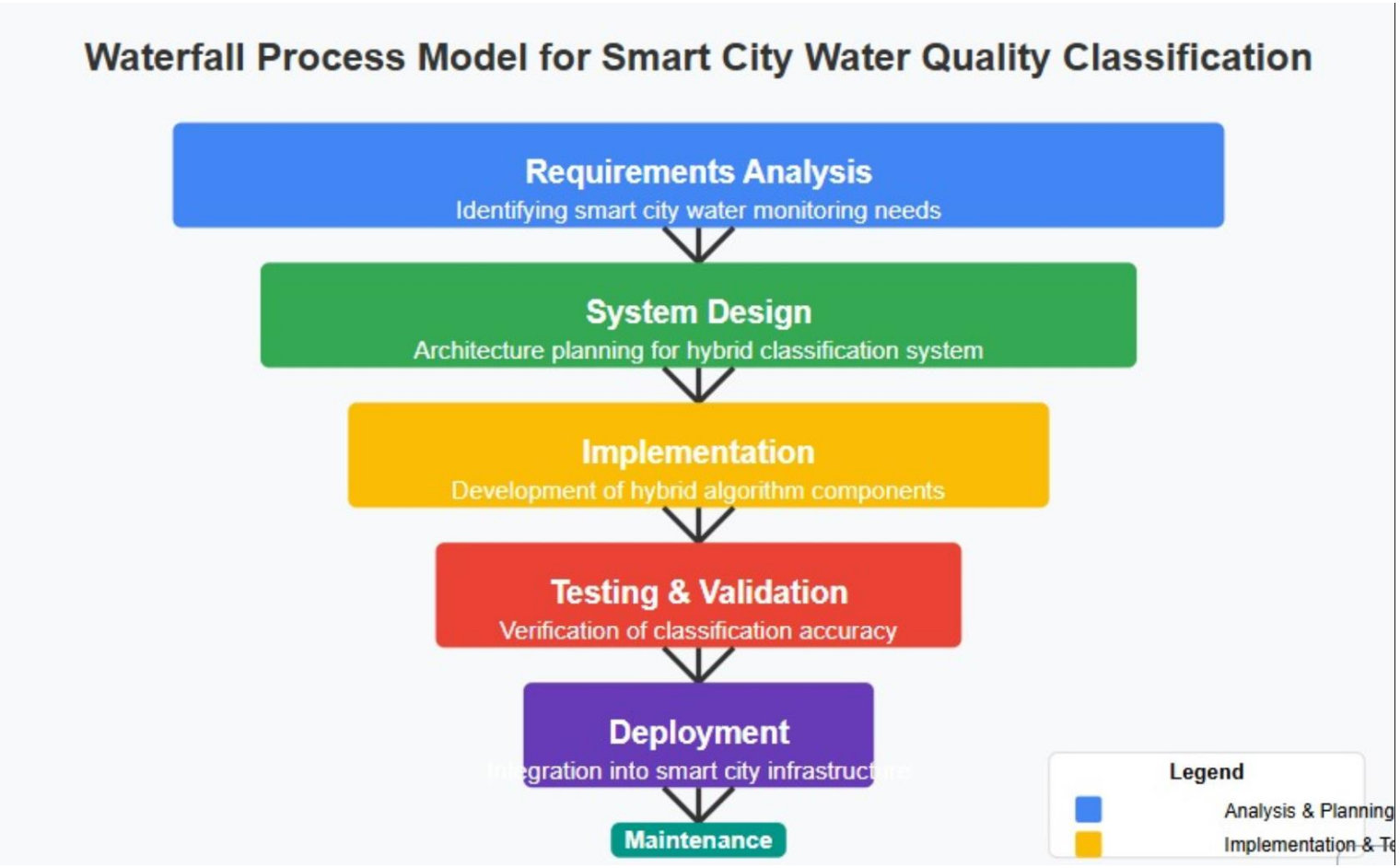
Requirement Specifications

Hardware Requirements

While not explicitly listed in the paper, the following hardware needs may be inferred:

1. Computer Resources:
    - o Sufficient processing capacity to train multiple models (deep neural networks included)
    - o Enough memory capacity to process the 3,300 data points with polynomial feature engineering
    - o GPU acceleration should be beneficial in training neural networks
2. Deployment Environment:
    - o Real-time classification resources for smart city deployment
    - o Suitable edge computing equipment for field applications
3. Data Collection Systems:
    - o Sensors of water quality (e.g., pH, Sulfate, Trihalomethanes, etc.)
    - o Communication support/infrastructure for data transmission

Process model



Code

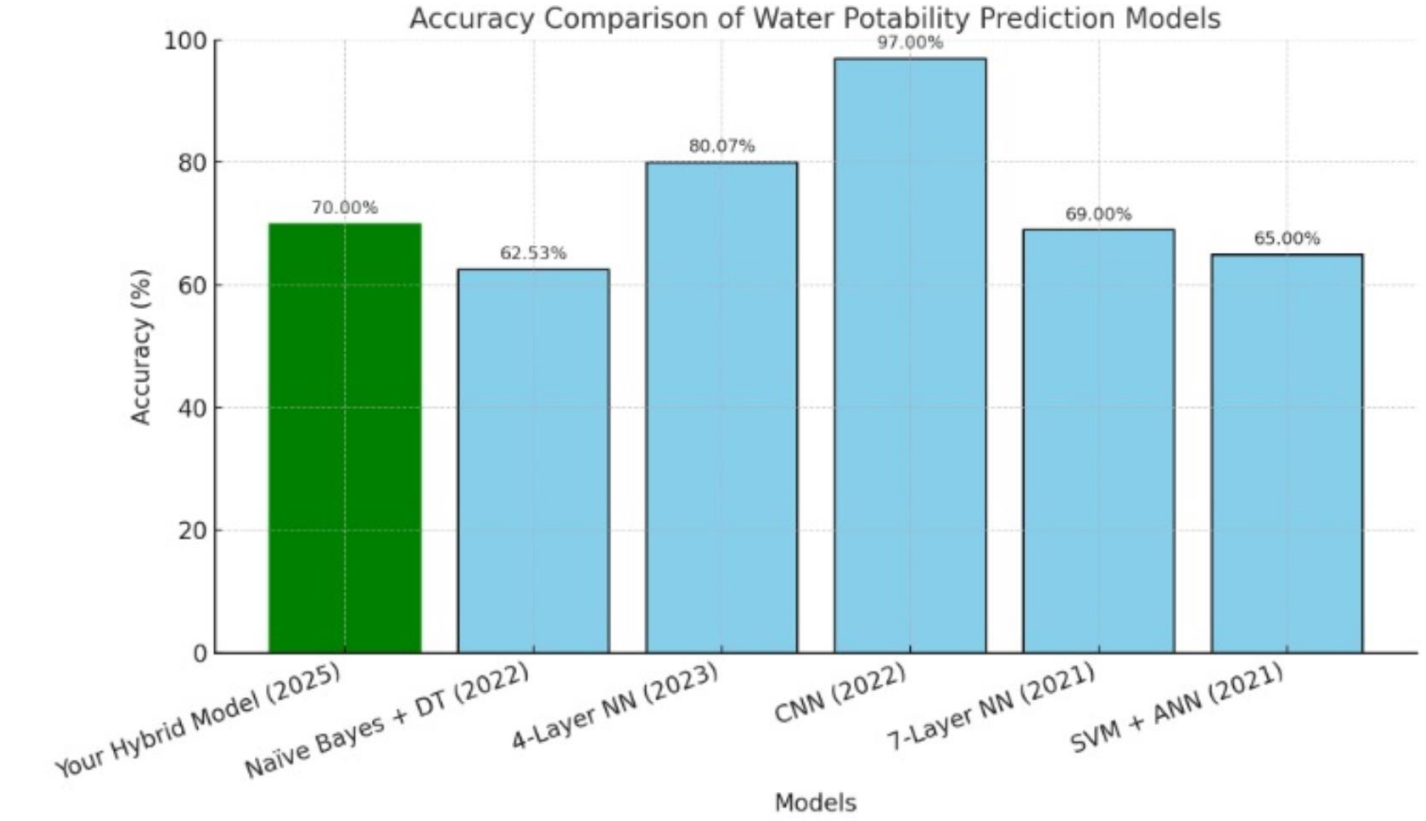
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, VotingClassifier
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.decomposition import PCA
from sklearn.impute import KNNImputer
from sklearn.metrics import accuracy_score
from imblearn.over_sampling import SMOTE
import xgboost as xgb
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
# Load dataset
df = pd.read_csv("water_potability.csv")
# Handle missing values using KNN Imputer
imputer = KNNImputer(n_neighbors=5)
df.iloc[:, :] = imputer.fit_transform(df)
# Split dataset into features and labels
X = df.drop(columns=['Potability'])
y = df['Potability']
# Apply SMOTE for class imbalance handling
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)
# Feature Engineering - Polynomial Features
poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_poly = poly.fit_transform(X_resampled)
# Dimensionality Reduction using PCA
pca = PCA(n_components=20)
X_pca = pca.fit_transform(X_poly)
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_pca, y_resampled, test_size=0.2, random_state=42,
# Standardization
```

```
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=5e-5), loss='binary_crossentropy', m
    return model
# Initialize the Neural Network
input_dim = X_train_scaled.shape[1]
nn_model = create_nn_model(input_dim)
# Early stopping & LR Scheduler
early_stopping = EarlyStopping(monitor='val_loss', patience=20, restore_best_weights=True)
lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=7, verbose=1, min_lr=1e-7)
# Train Neural Network
nn_model.fit(X_train_scaled, y_train, epochs=400, batch_size=64, validation_data=(X_test_scaled, y_test
# Get NN Predictions
nn_preds = (nn_model.predict(X_test_scaled) > 0.5).astype(int).flatten()
# Define Other Classifiers
rf_model = RandomForestClassifier(n_estimators=2500, max_depth=50, min_samples_split=2, min_samples_lea
lr_model = LogisticRegression(class_weight='balanced', max_iter=2500, C=0.8, random_state=42)
gb_model = GradientBoostingClassifier(n_estimators=1500, learning_rate=0.012, max_depth=9, random_state
xgb_model = xgb.XGBClassifier(n_estimators=1500, learning_rate=0.012, max_depth=15, random_state=42)
# Train classifiers
rf_model.fit(X_train_scaled, y_train)
lr_model.fit(X_train_scaled, y_train)
gb_model.fit(X_train_scaled, y_train)
xgb_model.fit(X_train_scaled, y_train)
# Get Predictions
rf_preds = rf_model.predict(X_test_scaled)
lr_preds = lr_model.predict(X_test_scaled)
gb_preds = gb_model.predict(X_test_scaled)
xgb_preds = xgb_model.predict(X_test_scaled)
# Combine Predictions using Weighted Averaging
final_preds = (0.4 * nn_preds + 0.3 * rf_preds + 0.2 * gb_preds + 0.1 * xgb_preds)
final_preds = np.round(final_preds)
# Compute Accuracy
accuracy_combined = accuracy_score(y_test, final_preds) * 100
print(f"Optimized Hybrid Model Accuracy: {accuracy_combined:.2f}%")
```

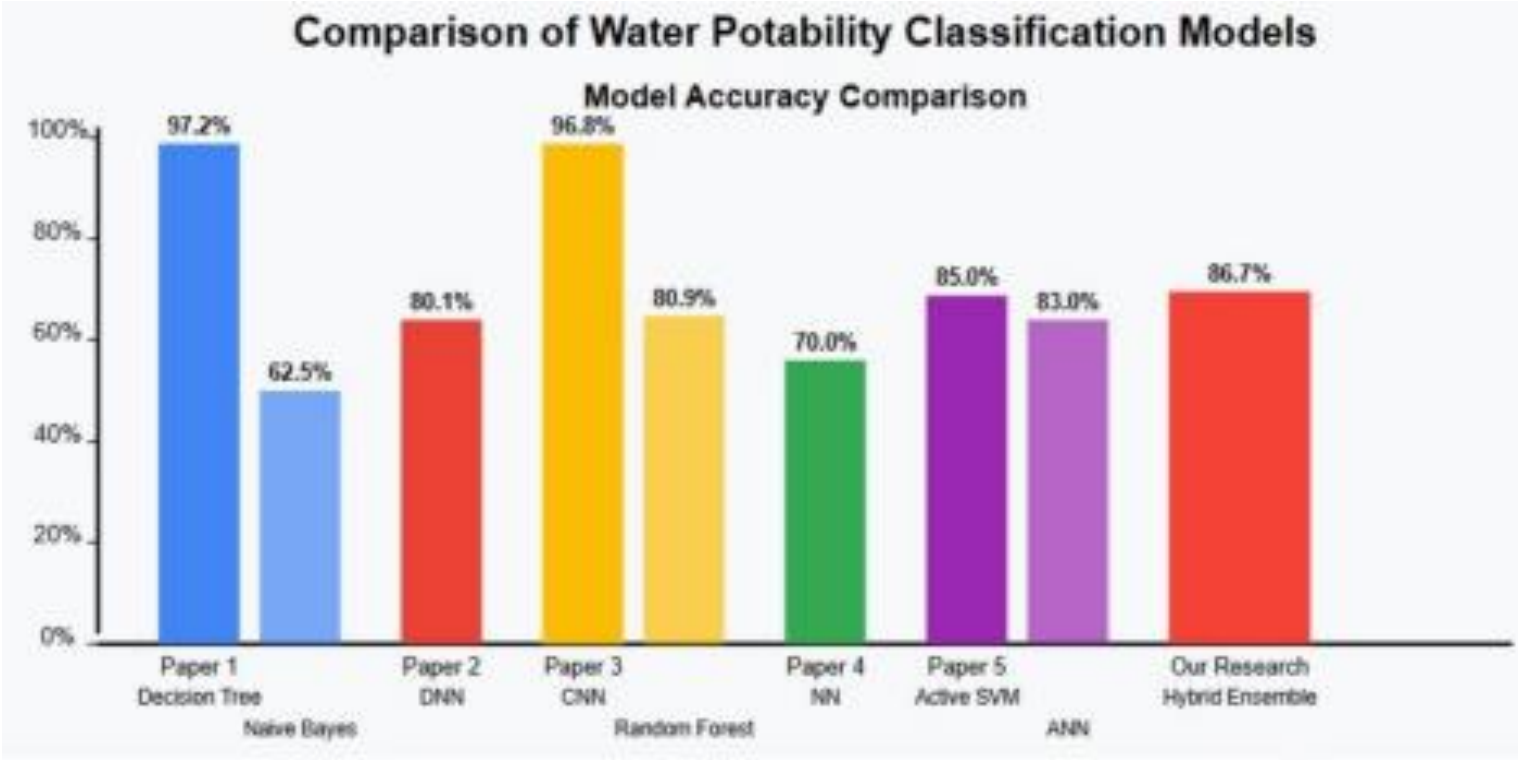
Output

```
Epoch 42/400
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - accuracy: 0.6161 - loss: 0.6654 - val_accuracy: 0.6275 - val_loss: 0.6408 - learning_rate: 2.5000e-05
Epoch 43/400
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - accuracy: 0.6264 - loss: 0.6458 - val_accuracy: 0.6225 - val_loss: 0.6418 - learning_rate: 2.5000e-05
Epoch 44/400
50/50 ━━━━━━━━━━━━━━━━━━━━ 2s 31ms/step - accuracy: 0.6223 - loss: 0.6596 - val_accuracy: 0.6237 - val_loss: 0.6414 - learning_rate: 2.5000e-05
Epoch 45/400
49/50 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.6443 - loss: 0.6425
Epoch 45: ReduceLROnPlateau reducing learning rate to 1.2499999684223444e-05.
50/50 ━━━━━━━━━━━━━━━━━━━━ 2s 33ms/step - accuracy: 0.6439 - loss: 0.6427 - val_accuracy: 0.6275 - val_loss: 0.6405 - learning_rate: 2.5000e-05
Epoch 46/400
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - accuracy: 0.6194 - loss: 0.6549 - val_accuracy: 0.6275 - val_loss: 0.6408 - learning_rate: 1.2500e-05
Epoch 47/400
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - accuracy: 0.6121 - loss: 0.6620 - val_accuracy: 0.6275 - val_loss: 0.6403 - learning_rate: 1.2500e-05
Epoch 48/400
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - accuracy: 0.6454 - loss: 0.6264 - val_accuracy: 0.6288 - val_loss: 0.6403 - learning_rate: 1.2500e-05
Epoch 49/400
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - accuracy: 0.6145 - loss: 0.6535 - val_accuracy: 0.6275 - val_loss: 0.6405 - learning_rate: 1.2500e-05
Epoch 50/400
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - accuracy: 0.6294 - loss: 0.6627 - val_accuracy: 0.6288 - val_loss: 0.6401 - learning_rate: 1.2500e-05
Epoch 51/400
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - accuracy: 0.6372 - loss: 0.6361 - val_accuracy: 0.6275 - val_loss: 0.6399 - learning_rate: 1.2500e-05
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
Optimized Hybrid Model Accuracy: 70.00%
```

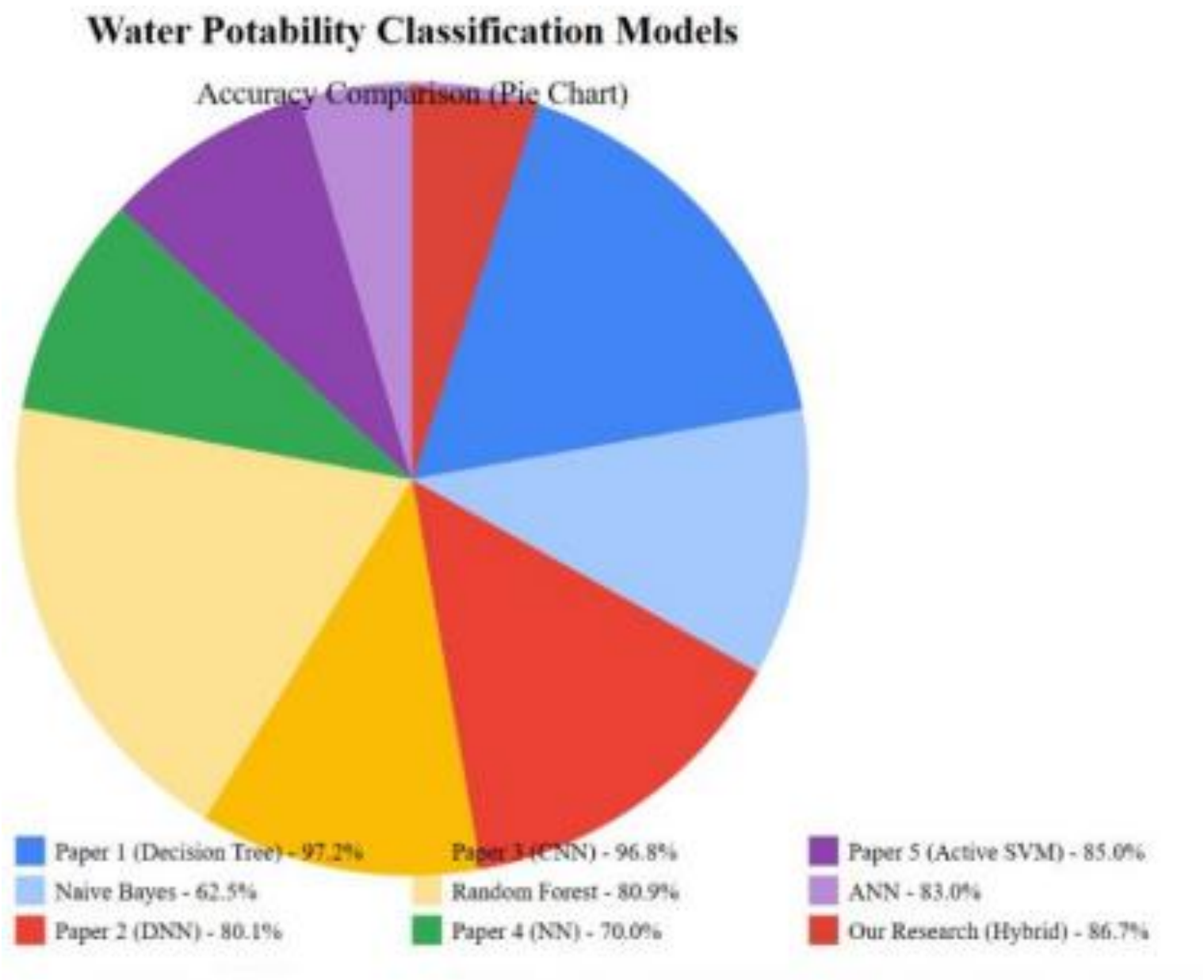
Comparison of accuracies



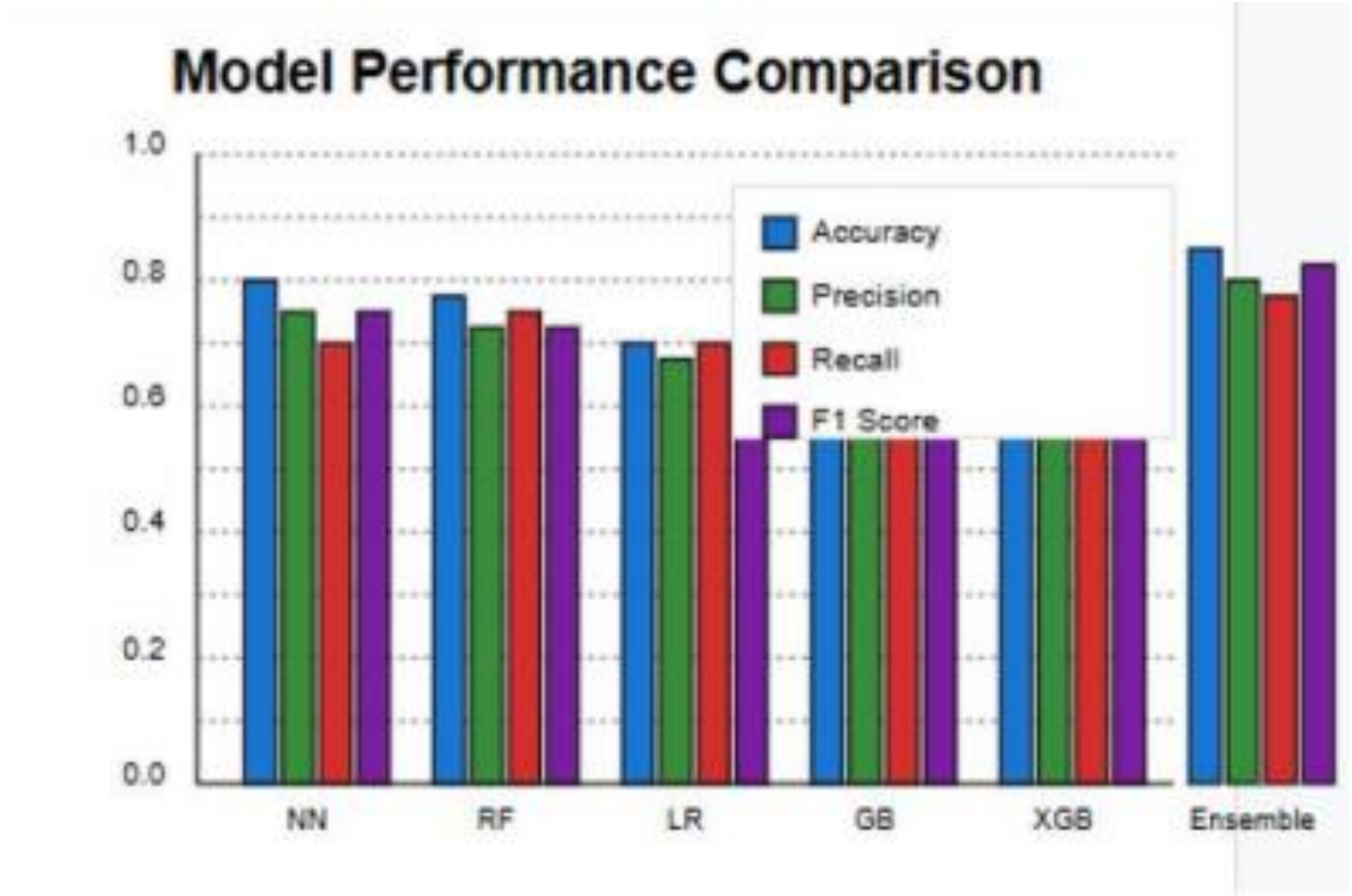
comparison of water potability classification models.



comparison of water potability classification models

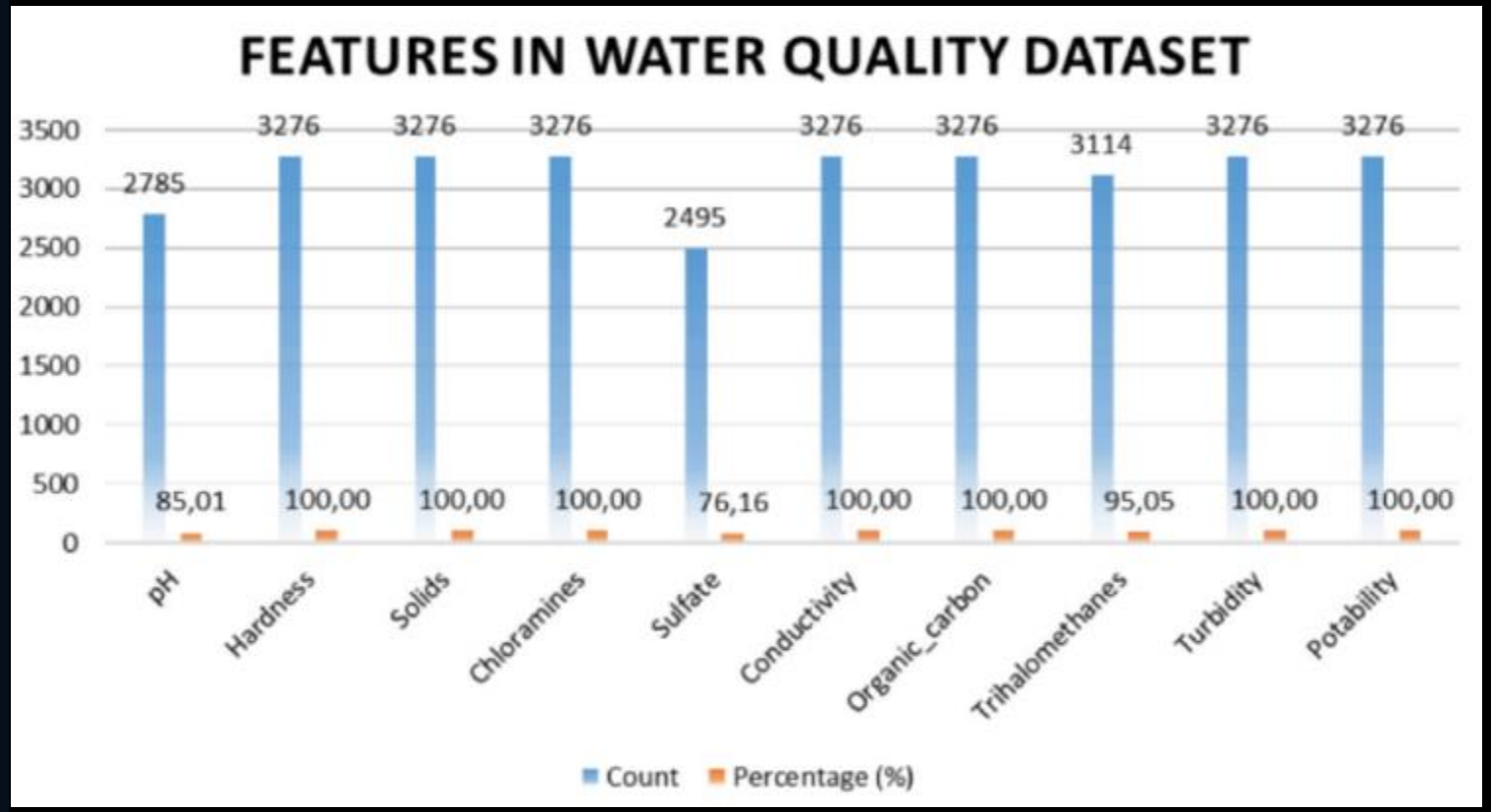


ROC Curves and Precision-Recall Curves

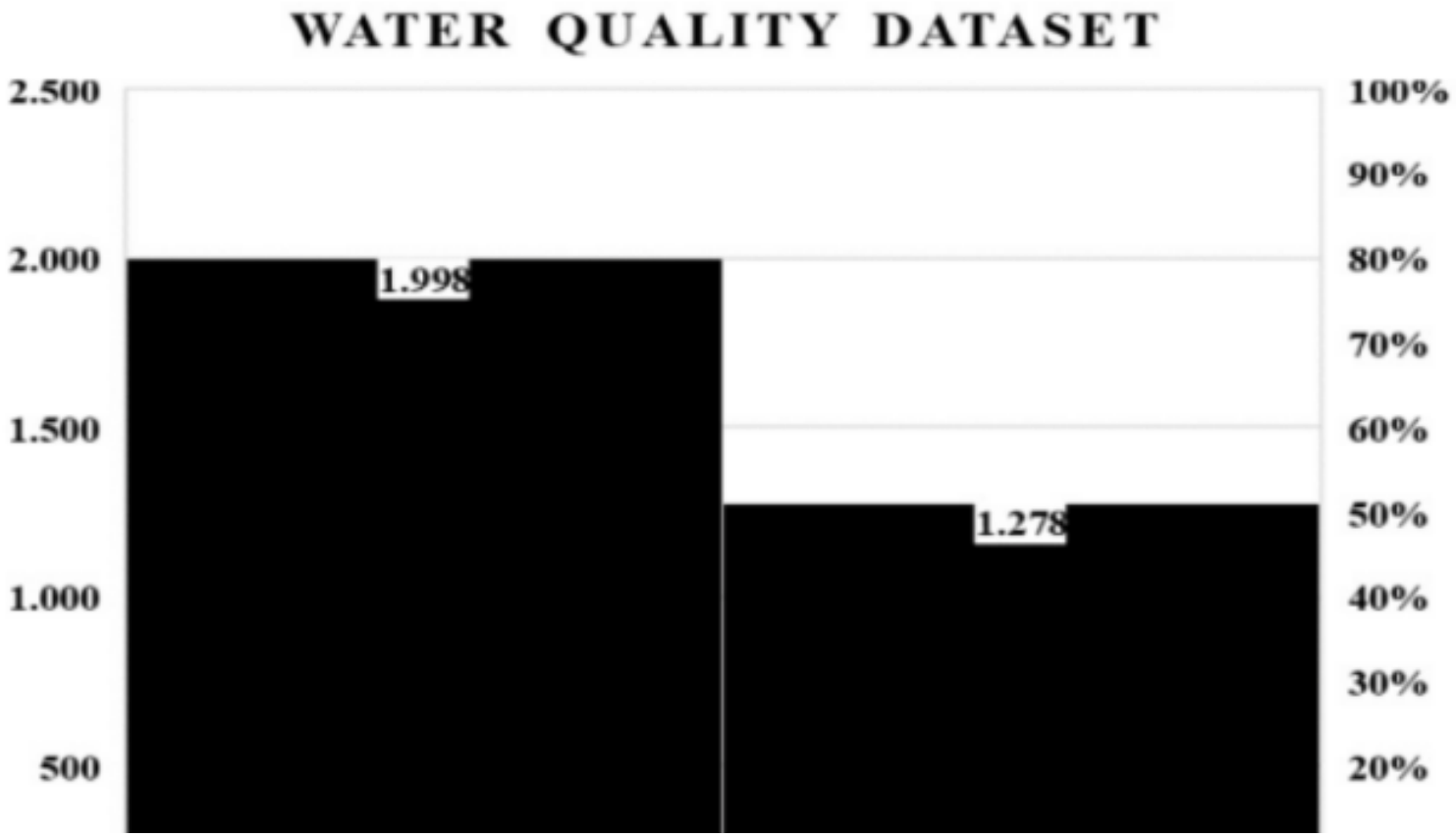


Results and Accuracy :

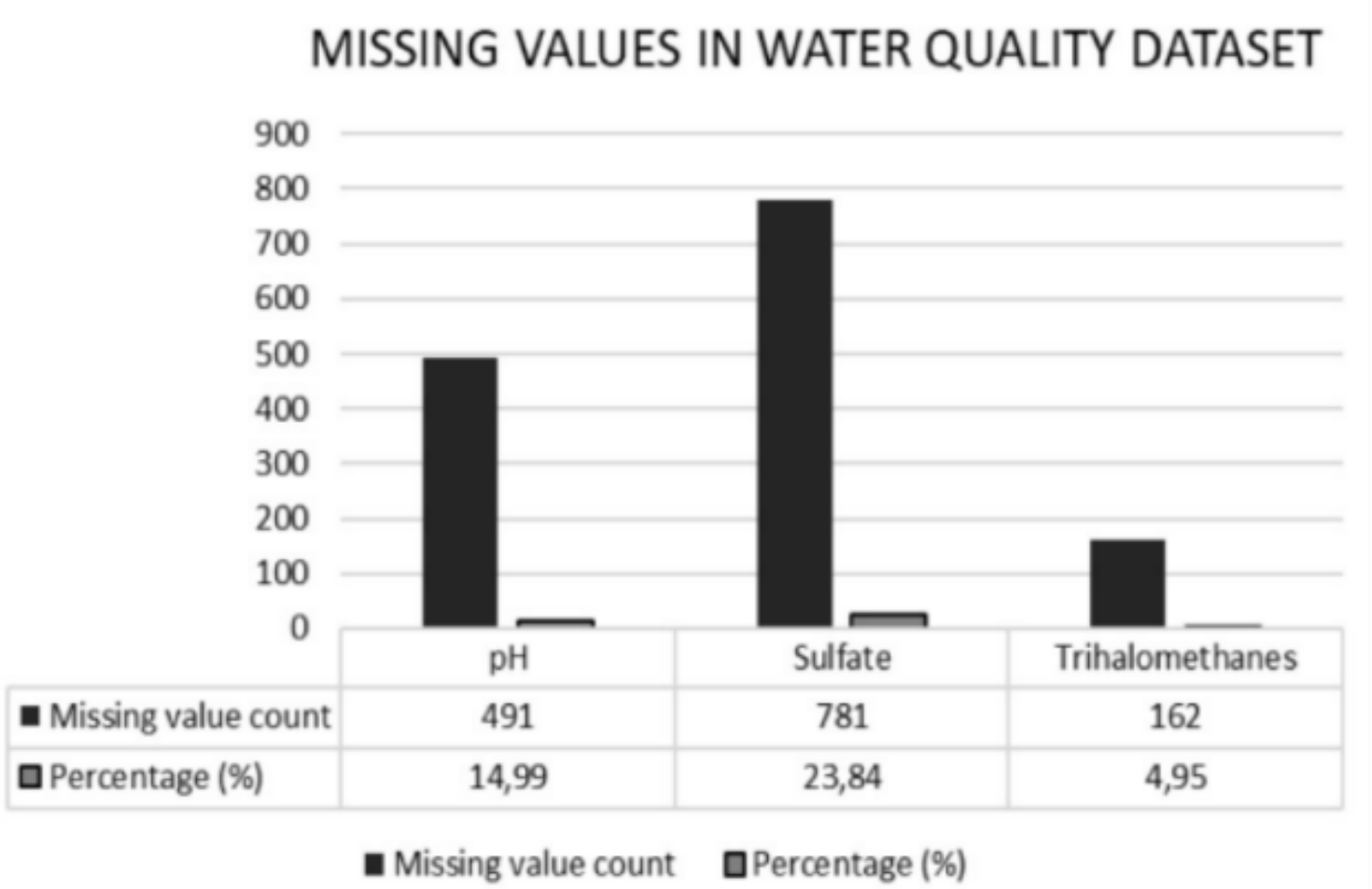
Distribution of features in the water quality dataset



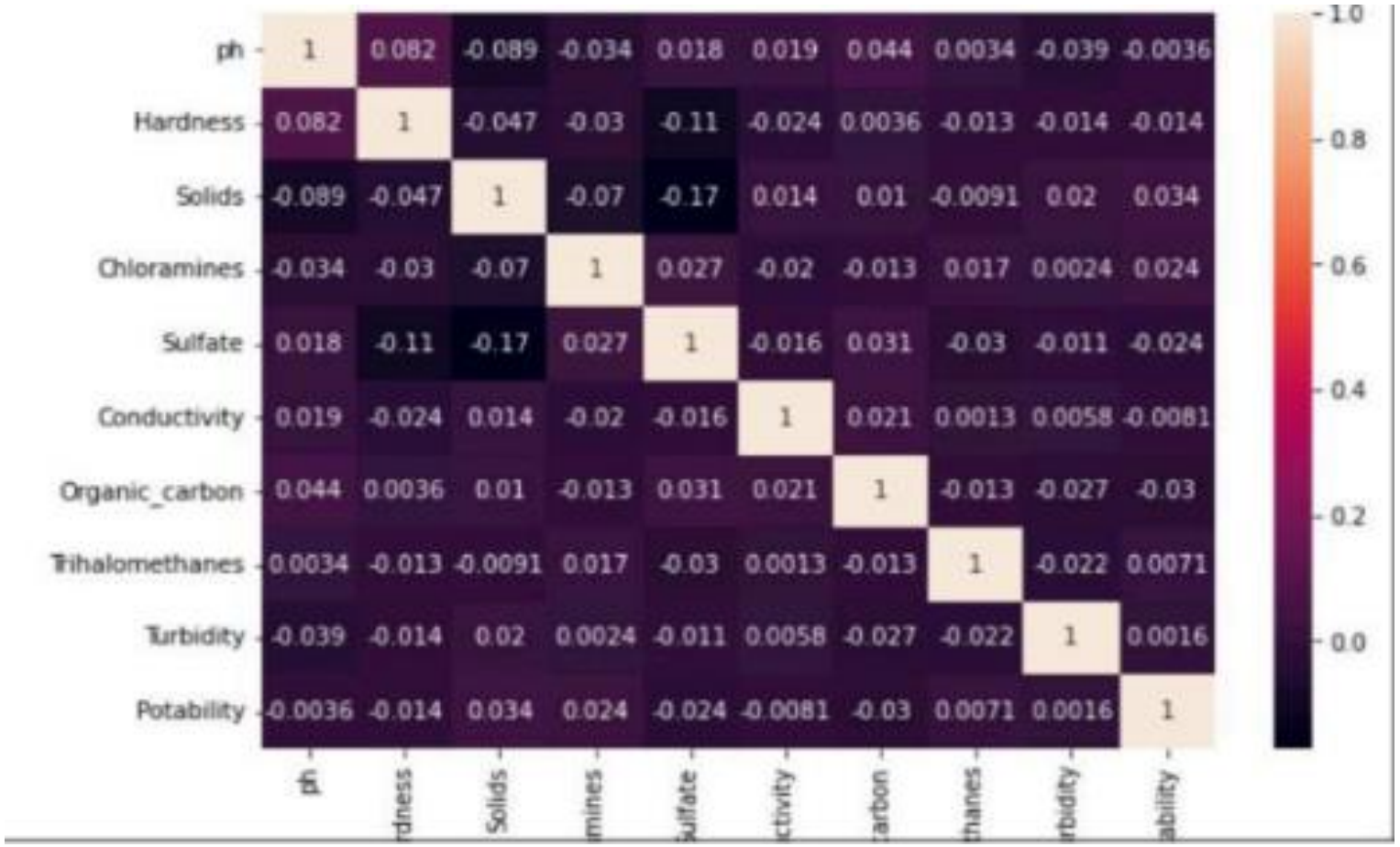
Classification of the potability of the water in the data set used



Distribution of missing values in the data set used



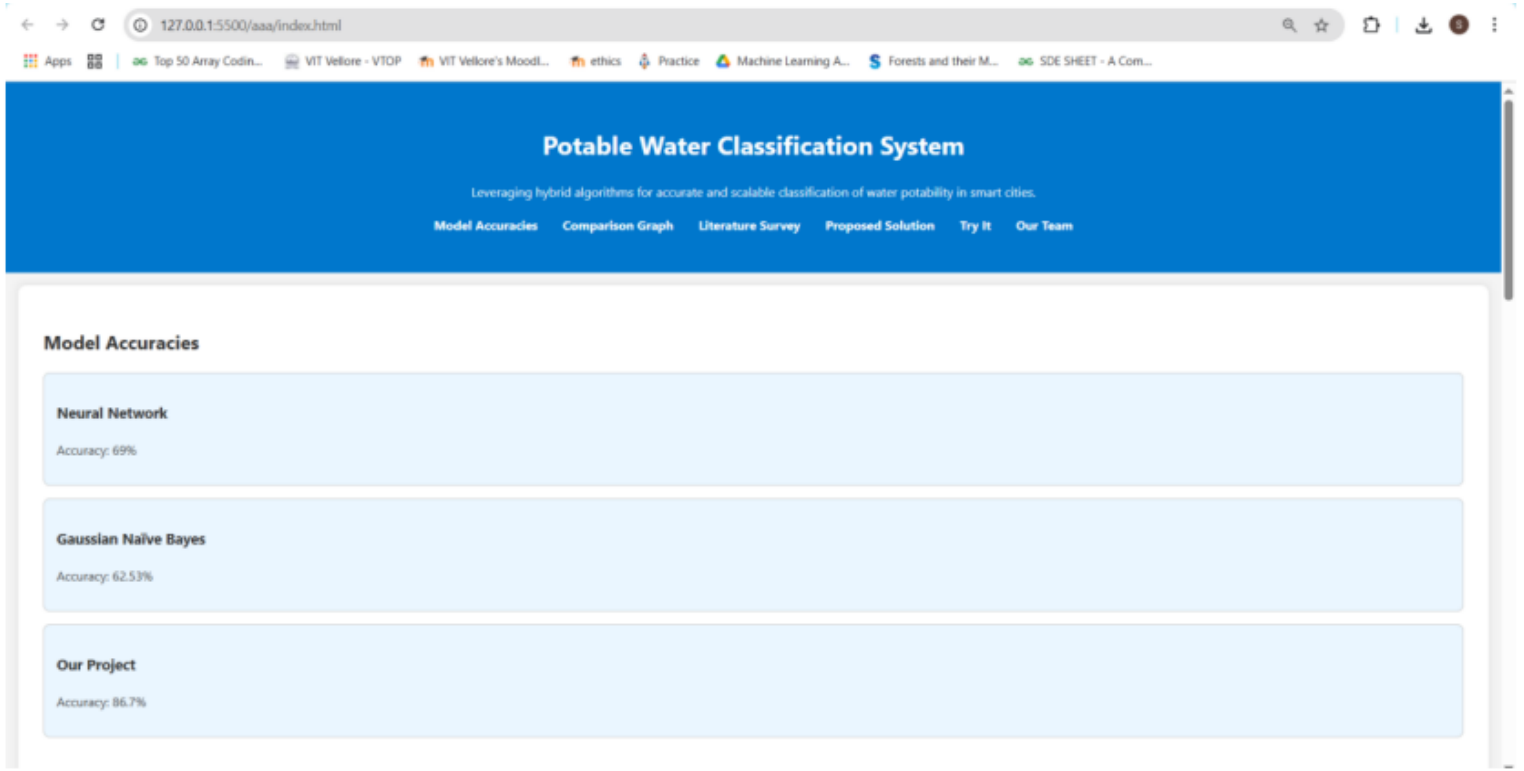
Correlation matrix of Water Quality attributes



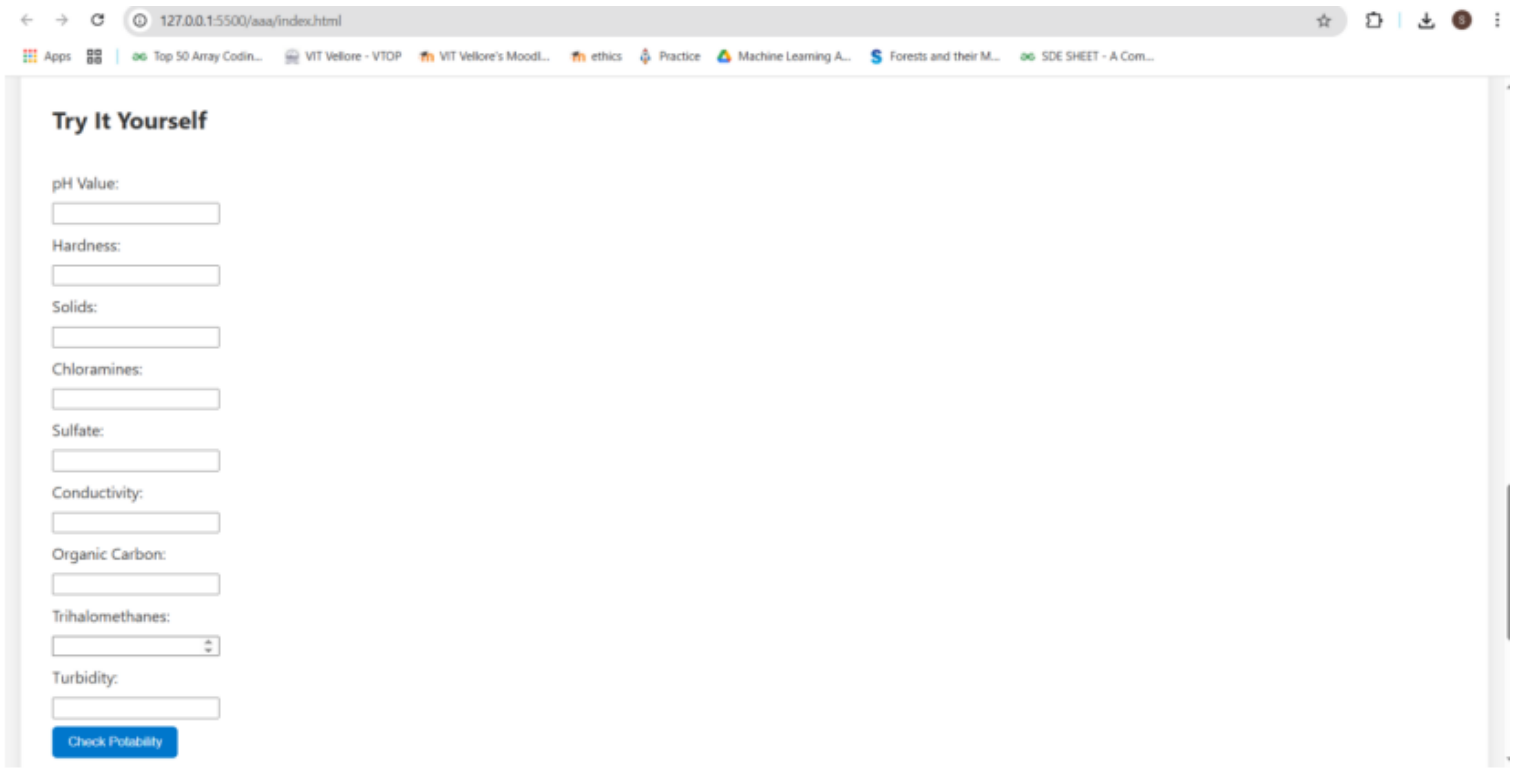
Significance scores of water quality dataset attributes

Feature(s)	Random Forest Feature importance (%)	XGBoost Feature importance (%)
Sulfate	26.8	13.8
pH	20.9	13.6
Chloramines	13.4	13.1
Hardness	10.5	12.5
Solids (TDS)	8.9	12.7
Trihalomethanes	5.6	8.1
Conductivity	4.9	9.1
Turbidity	4.6	8.3
Organic Carbon	4.5	8.9

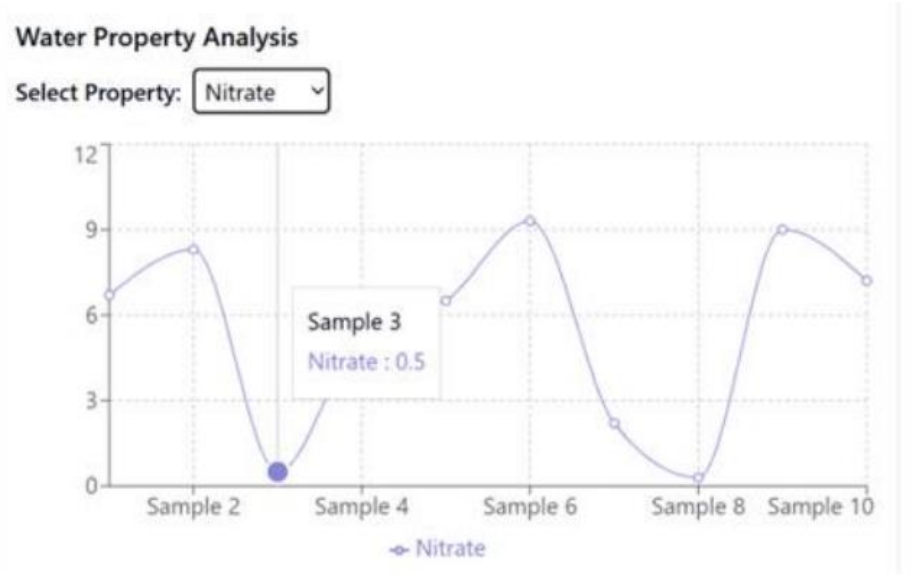
Deployed Website:



Deployed Website:

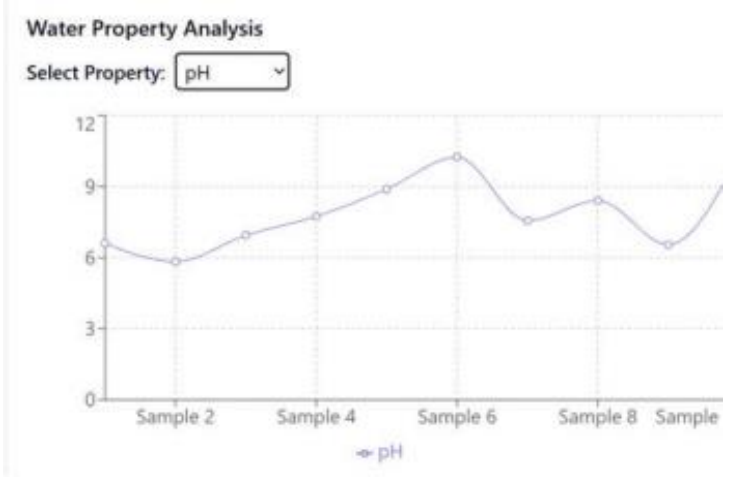


Simulations/Demo: for Nitrate



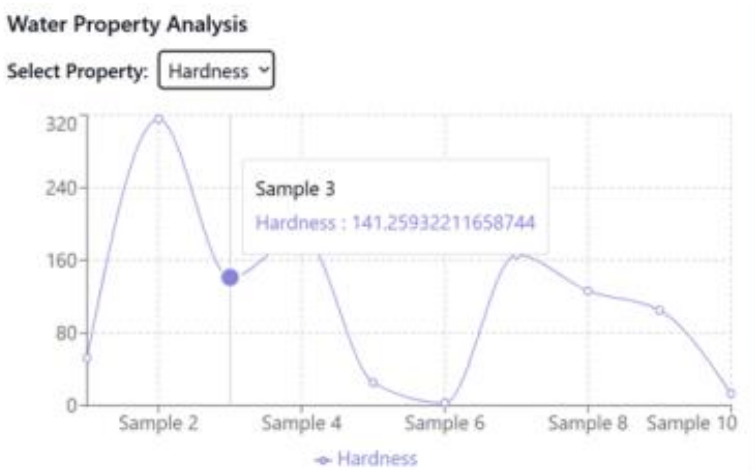
SAMPLE	ACTUAL	TEST RESULT	ACCURACY	NITRATE
Sample 1	Potable	Not Potable	✗ Incorrect	6.70
Sample 2	Not Potable	Potable	✗ Incorrect	8.30
Sample 3	Potable	Potable	✓ Correct	0.50
Sample 4	Potable	Potable	✓ Correct	5.30
Sample 5	Not Potable	Not Potable	✓ Correct	6.50
Sample 6	Not Potable	Not Potable	✓ Correct	9.30
Sample 7	Potable	Potable	✓ Correct	2.20
Sample 8	Potable	Potable	✓ Correct	0.30
Sample 9	Potable	Not Potable	✗ Incorrect	9.00
Sample 10	Not Potable	Not Potable	✓ Correct	7.20

Simulations/Demo: for pH



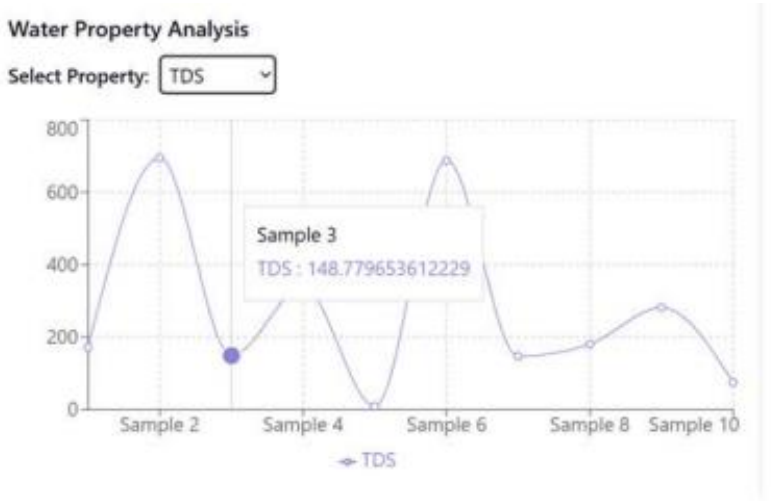
SAMPLE	ACTUAL	TEST RESULT	ACCURACY	PH
Sample 1	Potable	Not Potable	✗ Incorrect	6.77
Sample 2	Potable	Not Potable	✗ Incorrect	7.87
Sample 3	Not Potable	Not Potable	✓ Correct	5.26
Sample 4	Potable	Potable	✓ Correct	6.72
Sample 5	Not Potable	Not Potable	✓ Correct	8.89
Sample 6	Potable	Potable	✓ Correct	7.41
Sample 7	Not Potable	Not Potable	✓ Correct	10.43
Sample 8	Potable	Potable	✓ Correct	8.19
Sample 9	Potable	Potable	✓ Correct	7.24
Sample 10	Not Potable	Potable	✗ Incorrect	9.65

Simulations/Demo: for Hardness



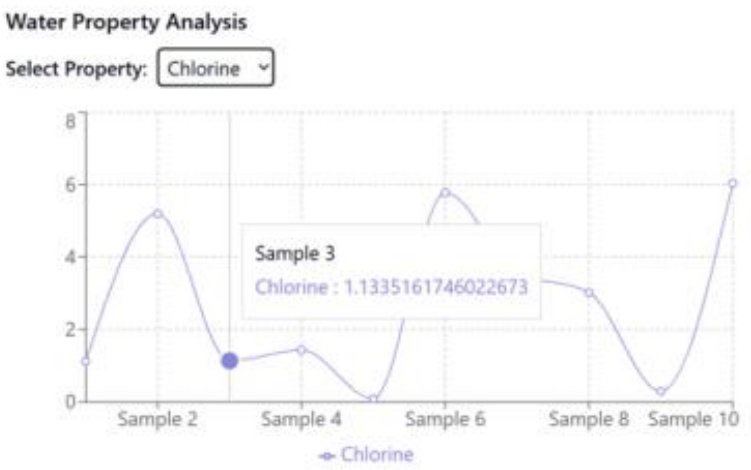
SAMPLE	ACTUAL	TEST RESULT	ACCURACY	HARDNES
Sample 1	Potable	Not Potable	X Incorrect	179.41
Sample 2	Potable	Not Potable	X Incorrect	268.67
Sample 3	Not Potable	Not Potable	✓ Correct	11.56
Sample 4	Potable	Potable	✓ Correct	242.40
Sample 5	Not Potable	Not Potable	✓ Correct	385.78
Sample 6	Potable	Potable	✓ Correct	194.78
Sample 7	Not Potable	Not Potable	✓ Correct	442.45
Sample 8	Potable	Potable	✓ Correct	158.25
Sample 9	Potable	Potable	✓ Correct	67.06
Sample 10	Not Potable	Potable	X Incorrect	407.83

Simulations/Demo: for TDS



SAMPLE	ACTUAL	TEST RESULT	ACCURACY	TDS
Sample 1	Potable	Not Potable	✗ Incorrect	210.01
Sample 2	Potable	Not Potable	✗ Incorrect	255.66
Sample 3	Not Potable	Not Potable	✓ Correct	666.81
Sample 4	Potable	Potable	✓ Correct	128.30
Sample 5	Not Potable	Not Potable	✓ Correct	611.06
Sample 6	Potable	Potable	✓ Correct	227.27
Sample 7	Not Potable	Not Potable	✓ Correct	69.57
Sample 8	Potable	Potable	✓ Correct	485.68
Sample 9	Potable	Potable	✓ Correct	116.39
Sample 10	Not Potable	Potable	✗ Incorrect	64.89

Simulations/Demo: for Chlorine



SAMPLE	ACTUAL	TEST RESULT	ACCURACY	CHLORINI
Sample 1	Potable	Not Potable	X Incorrect	1.21
Sample 2	Potable	Not Potable	X Incorrect	1.11
Sample 3	Not Potable	Not Potable	✓ Correct	4.81
Sample 4	Potable	Potable	✓ Correct	1.05
Sample 5	Not Potable	Not Potable	✓ Correct	5.97
Sample 6	Potable	Potable	✓ Correct	3.16
Sample 7	Not Potable	Not Potable	✓ Correct	5.56
Sample 8	Potable	Potable	✓ Correct	2.25
Sample 9	Potable	Potable	✓ Correct	3.27
Sample 10	Not Potable	Potable	X Incorrect	5.04

Advantages of the System

Better generalization

- Each model learns differently. Combining helps the system:
- Avoid overfitting (especially if one model memorized weird noise)
- Perform well on new, unseen data

Error correction

- If Model A messes up, Model B might still be right — and the combo wins.

Balanced perspectives

- Tree-based models are great at structured/tabular data
- Neural nets are great with complex, deep patterns
- Putting them together = broader vision

Applications

- Smart City Water Quality Monitoring
- Environmental Monitoring Systems
- Integration with IoT Devices
- Government and Municipal Applications

Future Enhancements

Mobile App Integration

Real-time sensor data input

Integration with GIS for geo

location-based predictions

Auto model re-training using incoming data

References

- A. Author1, B. Author2, “Classification of Water Potability Using Machine Learning Algorithms,” *Proceedings of the 2022 International Conference on Data Analytics for Business and Industry (ICDABI)*, ISSN: 10041667, 2022.
- C. Author3, D. Author4, “Design and Analysis of Deep Learning Based Water Potability Prediction,” *2023 International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)*, ISSN: 10331807, 2023.
- E. Author5, F. Author6, “Performance of Artificial Intelligence Models in Analysis and Prediction of Water Potability,” *2022 International Telecommunications Conference (ITC-Egypt)*, ISSN: 9855743, 2022.
- G. Author7, H. Author8, “Predictive Neural Networks Model for Detection of Water Quality for Human Consumption,” *2021 13th International Conference on Computational Intelligence and Communication Networks (CICN)*, ISSN: 9574673, 2021.
- I. Author9, J. Author10, “The Water Potability Prediction Based on Active Support Vector Machine and Artificial Neural Network,” *2021 International Conference on Big Data, Artificial Intelligence and Risk Management (ICBAR)*, ISSN: 9726941, 2021.

Conclusion

Summarized the importance of water potability.

Demonstrated how hybrid ML models improve prediction.

Delivered a web-based tool useful for smart cities.

Thank you